# BSC Computer Science

## University of London

## Singapore Institute of Management

## Intelligent Signal Processing

### End of Term Submission

**Aswathan K**

Exercise 1: Traffic Monitoring System ..... 4
Introduction: ..... 4
Frame Differencing & Background Subtraction: ..... 4
Alternative State of The Art Technique: ..... 6
Coursera Link: ..... 7
Exercise 2: Rice Coding for Lossless WAV Compression ..... 8
Introduction: ..... 8
Rice Coding & Delta Prediction: ..... 8
Analysis of Result & Further Implementations: ..... 9
Coursera Link: ..... 10
Exercise 3: Narbonne Film Festival Format Validator ..... 11
Introduction: ..... 11
FFmpeg and FFprobe Installation & Configuration: ..... 11
Application Analysis: ..... 11
Technical Terms: ..... 12
Coursera Link: ..... 12
Exercise 4: Emerging CV and Audio Applications in Retail and E-Commerce ..... 13
Three Emerging Computer Vision Applications: ..... 13
1. Cashierless checkout systems: ..... 13
2. Shelf Intelligence & Planogram Compliance: ..... 13
3. Visual Search & Multimodal Product Discovery: ..... 14
Two Popular or state-of-the-art Techniques: ..... 14
Technique 1: Vision Transformers (ViT and hierarchical variants): ..... 14
Technique 2: Real-time detection and tracking (YOLO plus ByteTrack): ..... 16
Two Creative & Novel Uses of CV Plus Audio Processing: ..... 18
Use Case 1: Accessibility assistant for elderly and low-vision shoppers: ..... 18
Use Case 2: Audiovisual checkout anomaly prevention: ..... 18
References: ..... 19

## Figures & Tables:

Figure 1: Vehicle detection with bounding boxes under normal traffic conditions ..... 5
Figure 2: Contour merging limitation observed when two vehicles overlap ..... 6
Figure 3: Encoding pipeline ..... 8
Figure 4: Decoding pipeline ..... 8
Figure 5: ViT Pipeline ..... 15
Figure 6: YOLOv8 + ByteTrack Pipeline ..... 17

Table 1: No of cars detected going downtown & cpm rate ..... 7
Table 2: Rice coding compression results without Delta Prediction ..... 9
Table 3: Rice coding compression results with Delta Prediction ..... 9

# Exercise 1: Traffic Monitoring System

## Introduction:

The developed application successfully detects and counts vehicles heading toward downtown Laramie using a computer vision pipeline built with OpenCV in Python. Task 1.1 implements a TrafficDetector class that isolates a defined interested region (IR) covering the main street, applies background subtraction to detect moving vehicles, and draws bounding boxes around detected contours. Task 1.2 extends this with a centroid-based tracking system within the analyseVideo function that assigns unique IDs to each detected vehicle, maintains a position history of the last five frames, and computes the average horizontal displacement (avg_dx) to filter only downtown-bound vehicles. A counting zone is defined at coordinates (840, 200, 1010, 345) where only cars with avg_dx < 1 are counted which effectively solves the problem of opposite coming cars from downtown being counted. Cars detected outside the zone are displayed with blue bounding boxes while cars inside are shown in green, providing clear visual feedback during processing.

## Frame Differencing & Background Subtraction:

Frame differencing is a motion detection technique that computes the pixel-wise absolute difference between consecutive frames. Moving objects produce non-zero-pixel values in the difference image while static backgrounds produce near-zero values, enabling foreground segmentation. It is simple and computationally efficient but sensitive to noise and lighting changes.

Background subtraction is a more robust extension where a model of the static background is built over time, and each new frame is compared against this model. The K-Nearest Neighbours (KNN) method used in this application classifies each pixel as foreground or background by comparing it against a set of recently observed pixel values. KNN is particularly effective for scenes with sudden object appearances, making it well-suited for traffic monitoring. Parameters such as history and dist2Threshold control how quickly the background model adapts and how sensitive the detection is, respectively. Both techniques are complemented by Gaussian blur for noise reduction and morphological closing to fill gaps in detected shapes.

A known limitation of the contour-based detection approach is that when two vehicles travel in close proximity, their foreground masks merge into a single contour, causing them to be detected as one object. This results in under-counting and is an inherent constraint of pixel-level background subtraction methods like KNN, which lack object-level understanding. Future work could address this using deep learning detectors such as YOLO, which can distinguish individual vehicles even under occlusion [27].

*Figure 1: Vehicle detection with bounding boxes under normal traffic conditions*

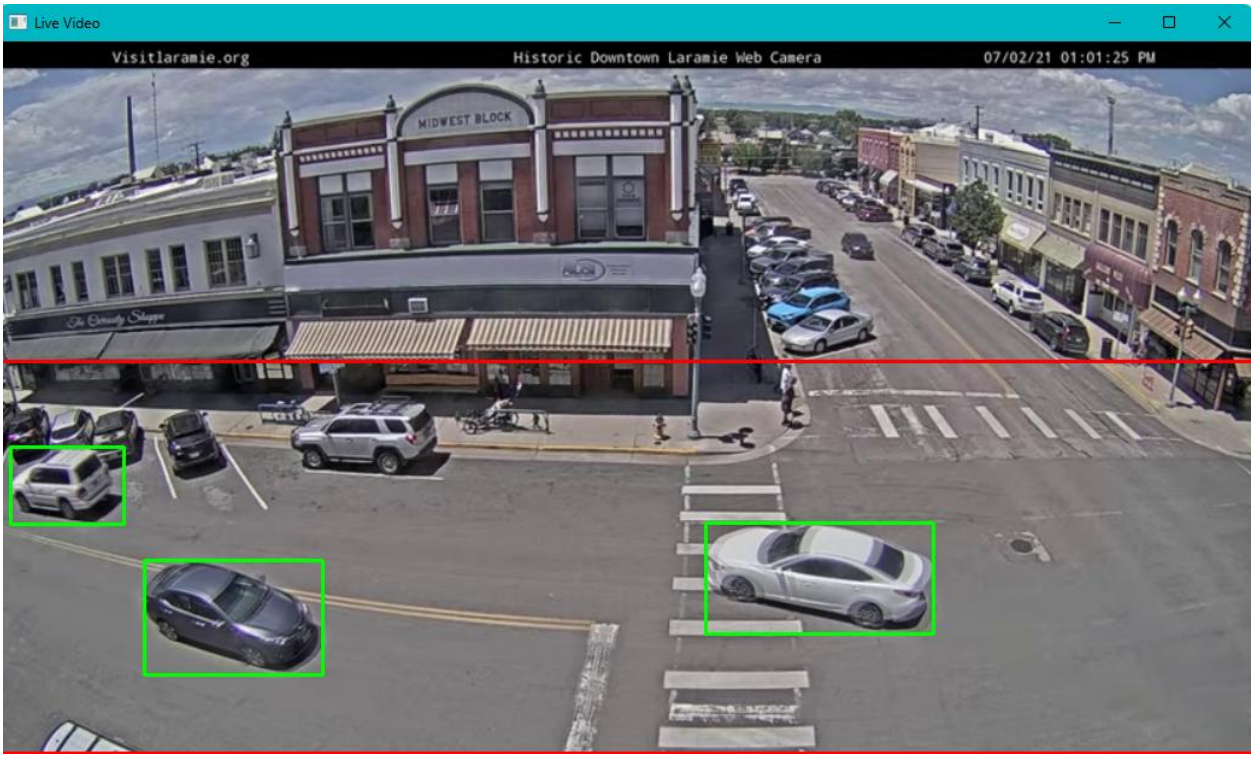
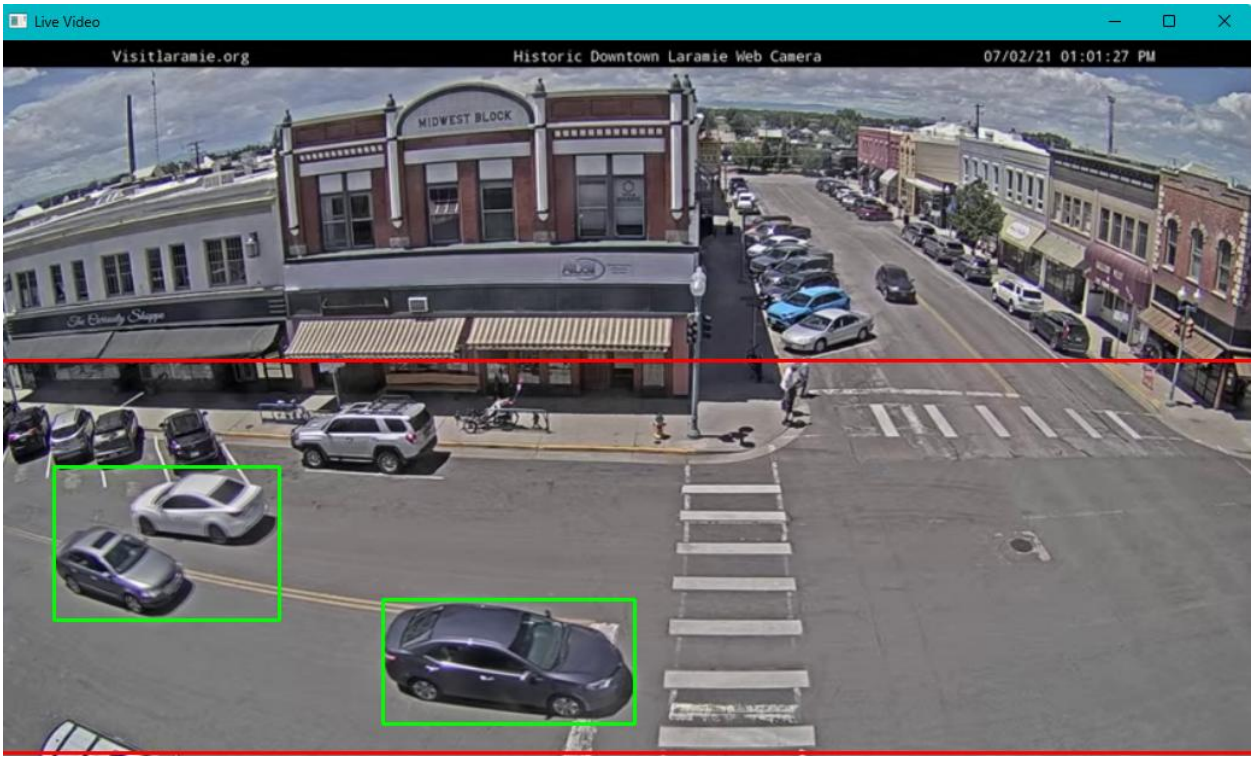


*Figure 2: Contour merging limitation observed when two vehicles overlap*




## Alternative State of The Art Technique:

A strong alternative to background subtraction for vehicle detection and tracking is YOLOv8 (You Only Look Once) combined with DeepSORT tracking [29]. YOLOv8 is a deep learning-based object detector that identifies vehicles directly from raw frames using a convolutional neural network trained on large datasets, eliminating the need for manual interested region definition or background modelling [28]. DeepSORT extends this by assigning persistent IDs to tracked objects using appearance features and Kalman filtering, making it significantly more robust to occlusion and varying lighting conditions compared to centroid-based tracking. This approach would greatly improve the application's accuracy in complex scenarios such as overlapping vehicles, shadows, and varying traffic density limitations that affect the current KNN-based pipeline.

*Table 1: No of cars detected going downtown & cpm rate*

| | Total number of cars | Cars per minute |
|---|---|---|
| Traffic_Laramie_1.mp4 | 10 | 3.37 |
| Traffic_Laramie_2.mp4 | 4 | 2.27 |

## Coursera Link:

https://hub.labs.coursera.org:443/connect/sharedmymkqlyx?forceRefresh=false&sessionMigrationMode=shadow

# Exercise 2: Rice Coding for Lossless WAV Compression

## Introduction:

The application encodes and decodes 16-bit PCM WAV files through a structured pipeline: reading mono audio samples, applying first-order delta (DPCM) prediction to generate small residuals, zigzag-mapping signed residuals to nonnegative integers, Rice-coding the result with fixed parameters K=4 and K=2, and serialising the output into a custom binary .ex2 container format. Decoding reverses all steps exactly, and SHA-256 hash comparison between the original and reconstructed WAV files confirms byte-perfect lossless reconstruction for every tested configuration. The application is structured into modular functions including read_wav_mono, delta_encode, rice_encode, encode_wav_to_ex2, rice_decode, delta_decode, and decode_ex2_to_wav, each handling a distinct stage of the pipeline cleanly.

*Figure 3: Encoding pipeline*

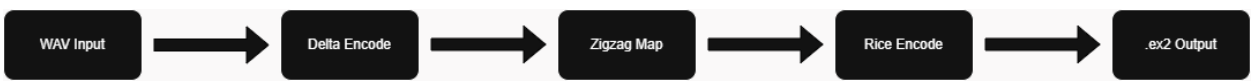



*Figure 4: Decoding pipeline*

## Rice Coding & Delta Prediction:

Rice coding is a specialised form of Golomb coding that encodes a nonnegative integer by splitting it into a quotient q = $\lfloor u \,/\, 2^k \rfloor$ and remainder r = $u \bmod 2^k$. The quotient is stored in unary format (q ones followed by a zero delimiter) and the remainder in K fixed bits, giving a compact representation when most values of u is small. Since raw PCM audio samples span the full int16 range (−32,768 to +32,767), direct Rice coding would produce enormous unary quotients and expand the file. Delta encoding addresses this by replacing each sample with its difference from the previous sample, exploiting the temporal correlation in audio. These residuals are typically concentrated near zero, making them well-suited for Rice coding. Zigzag mapping then converts signed residuals to nonnegative integers (mapping 0→0, −1→1, 1→2, −2→3, ...) before Rice coding is applied. The complete encode and decode cycle is verified lossless for both files across both K values.

*Table 2: Rice coding compression results without Delta Prediction*

| | Original size | Rice (K = 4 bits) | Rice (K = 2 bits) | % Compression (K = 4 bits) | % Compression (K = 2 bits) |
|---|---|---|---|---|---|
| Sound1.wav | 1002088 | 6013569 | 23069260 | -500.10% | -2202.12% |
| Sound2.wav | 1008044 | 39102991 | 155435731 | -3779.10% | -15319.54% |

*Table 3: Rice coding compression results with Delta Prediction*

| | Original size | Rice (K = 4 bits) | Rice (K = 2 bits) | % Compression (K = 4 bits) | % Compression (K = 2 bits) |
|---|---|---|---|---|---|
| Sound1.wav | 1002088 | 857473 | 2429927 | 14.43% | -142.49% |
| Sound2.wav | 1008044 | 56448285 | 224816526 | -5499.78% | -22202.25% |

## Analysis of Result & Further Implementations:

The compression rates differ significantly between the two files due to differences in their residual distributions after delta prediction. Sound1.wav achieves genuine compression of 14.43% at K=4, indicating that its delta residuals are small and tightly concentrated near zero, closely matching the geometric distribution that Rice coding assumes. Sound2.wav, by contrast, expands catastrophically at both K values, implying that its residuals are large and widely distributed, likely due to highly transient or noisy audio content. When residuals are large, the unary quotient q becomes very long, producing codewords far longer than the 16-bit original sample, causing severe expansion rather than compression.

The choice of K also significantly affects results. K=2 performs worse than K=4 for both files because a smaller $M = 2^k = 4$ means larger quotients for the same residual value, generating longer unary codes. K=4 with $M = 16$ distributes bits more efficiently across the quotient and remainder for moderately sized residuals, which is why Sound1 compresses at K=4 but not at K=2. The negative compression percentages are not errors. They correctly reflect that the encoded .ex2 file is larger than the original WAV when the residual distribution is unsuitable for Rice coding.

The application can be improved by implementing adaptive K selection per block, where the audio is split into short windows and the optimal K is chosen per block such as the approach used by the FLAC lossless audio codec.

## Coursera Link:

https://hub.labs.coursera.org:443/connect/sharedmymkqlyx?forceRefresh=false&sessionMigrationMode=shadow

# Exercise 3: Narbonne Film Festival Format Validator

## Introduction:

The developed application automates the format validation and conversion pipeline for the Narbonne Online Film Festival. The application uses ffprobe to extract metadata from submitted video files and ffmpeg to re-encode non-compliant submissions automatically. Task 3 implements a validation system that compares nine format parameters against the festival requirements stored in target_specs_required. Task 3.2 generates a written audit report (format_audit_report.txt) listing each file's compliance status and all problematic fields. Task 3.3 automatically re-encodes non-compliant files using ffmpeg with precise conversion parameters and saves them as *_formatOK.mp4 inside outputs/converted/. A final verification step re-probes every converted file to confirm full compliance, ensuring no manual intervention is required.

## FFmpeg and FFprobe Installation & Configuration:

On my Windows machine, ffmpeg and ffprobe were installed using Chocolatey. After installing Chocolatey via PowerShell in Administrator mode, I executed choco install ffmpeg, which automatically downloaded the binaries and added them to the system PATH. This made both tools globally accessible from any terminal or Python subprocess. The application verifies installation at runtime by running ffmpeg -version using subprocess.run() and checking the return code. For macOS users the application supports automatic installation via Homebrew (brew install ffmpeg), and for Linux via apt-get install ffmpeg.

## Application Analysis:

The application follows a four-stage pipeline: metadata extraction, validation, reporting, and conversion. The probe_file() function calls ffprobe with JSON output (-of json) to retrieve structured stream and format metadata. The validate_file() function parses this JSON and performs nine validation checks against target_specs_required, using helper functions to normalize ffprobe's output — such as converting fractional frame rates like 30000/1001 to 29.97 and simplifying pixel dimensions to aspect ratios using GCD. The convert_file() function constructs an ffmpeg command applying video filters for resolution and frame rate correction, H.265 encoding with controlled bitrate, and AAC stereo audio encoding. This architecture ensures the pipeline is repeatable and fully automated for large-scale submissions.

## Technical Terms:

- Video format (container): The file wrapper storing video, audio, and metadata streams. MP4 is widely compatible across devices and streaming platforms.
- Video codec: The algorithm used to compress video data. H.265 (HEVC) reduces file sizes by ~50% compared to H.264 while maintaining quality.
- Audio codec: The compression method for audio streams. AAC delivers better quality than MP3 at equivalent bitrates and is the streaming industry standard.
- Frame rate: The number of frames displayed per second. 25 FPS is the European PAL broadcast standard ensuring smooth motion playback.
- Aspect ratio: The proportional width-to-height relationship. 16:9 is the universal widescreen standard matching modern displays.
- Resolution: The pixel dimensions of each frame. 640×360 balances web streaming quality with minimal bandwidth usage.
- Video bit rate: The data volume processed per second for video. The 2–5 Mb/s range balances quality and file size.
- Audio bit rate: The data rate for the audio stream. Up to 256 kb/s ensures high-fidelity stereo sound without inflating file sizes.
- Audio channels: The number of independent audio tracks. Stereo (2 channels) provides spatial left-right separation standard for film audio.

## Coursera Link:

https://hub.labs.coursera.org:443/connect/sharedmymkqlyx?forceRefresh=false&sessionMigrationMode=shadow

# Exercise 4: Emerging CV and Audio Applications in Retail and E-Commerce

## Three Emerging Computer Vision Applications:

### 1. Cashierless checkout systems:

Cashierless stores are one of the clearest examples of computer vision moving from lab settings to real operational environments [1] [2]. The core business goal is to remove checkout friction by detecting what a customer picks up, keeps, or returns to the shelf. Typical system design combines ceiling cameras, shelf cameras, multi-object tracking, person re-identification, hand-object interaction recognition, and event-level temporal fusion. The model stack must resolve edge cases such as occlusions between customers, fast hand motion, and visually similar items. From a technical perspective, this application is relevant because it requires robust tracking over long time windows and across cameras. In controlled demos, performance can appear high, but real stores add variability in lighting, crowd density, clothing texture, and product packaging changes. Practical deployment therefore needs continual calibration and model to update pipelines. Its strategic value is large, reduced queue time, more stable throughput at peak hours, and potentially lower labor concentration at checkout. However, operational acceptance depends on trust, explainability of flagged events, and clear dispute resolution when model confidence is low.

### 2. Shelf Intelligence & Planogram Compliance:

Shelf intelligence focuses on product availability, quality, and planogram compliance. Retailers install fixed aisle cameras or robot-based scanners to capture shelf images and run detection plus classification models against expected shelf layouts [3], [4]. Instead of waiting for manual audits, teams can receive alerts for stockouts, misplaced products, and promotional placement failures in real time. This area is especially important because out-of-stock conditions create direct revenue loss and damage customer experience. In many stores, manual shelf checks are periodic and incomplete, so issues can persist for hours. Automated CV can reduce that latency to minutes. The engineering challenge is long-tail recognition: many SKUs differ only by small visual cues such as flavor text or color accents. Domain adaptation is also needed because camera angle, shelf reflection, and packaging redesign can break brittle models. A robust system pairs CV outputs with inventory and POS data so that operators can prioritize high-impact corrections rather than reacting to every low-confidence anomaly.

### 3. Visual Search & Multimodal Product Discovery:

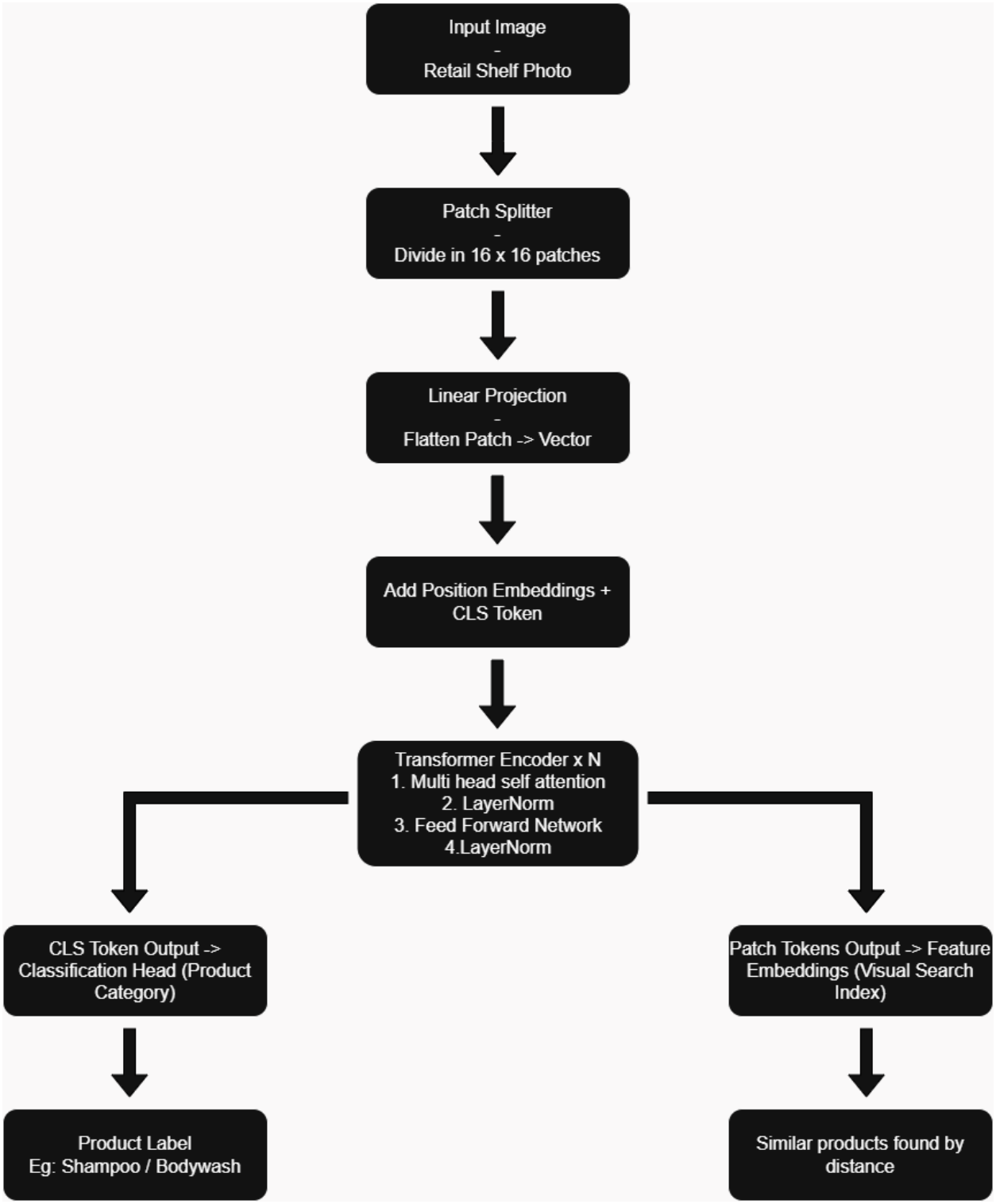
Visual search in e-commerce helps users find products from photos when text queries are vague. A shopper may upload an image of a shoe, lamp, or dress and expect similar results by style, color, and shape. Modern systems use embedding-based retrieval pipelines, often with multimodal encoders that align text and image representations [5], [6]. This application is highly relevant in categories where natural language is insufficiently precise. For example, a user may know the look they want but not the exact brand, pattern term, or product taxonomy. Visual retrieval bridges this gap and supports discovery across long-tail catalogs that are hard to navigate through keyword search alone. Technically, visual search requires strong feature extraction, efficient indexing, and ranking that balances visual similarity with business constraints such as availability, margin, and delivery region. A production-quality solution should also include bias checks, so retrieval quality is not disproportionately worse for specific demographic styles, lighting conditions, or skin tones in fashion imagery.

## Two Popular or state-of-the-art Techniques:

### Technique 1: Vision Transformers (ViT and hierarchical variants):

Vision Transformers and hierarchical transformer families such as Swin are now central to modern CV pipelines [7], [8]. Unlike local convolution kernels alone, transformer attention can capture broader context, which is valuable in cluttered retail scenes where object boundaries and interactions matter. These models are widely used as backbones for detection, segmentation, and retrieval. In retail, stronger context modeling helps dense shelf analysis and scene understanding where objects overlap or are partially visible. For e-commerce imagery, transformer-based embeddings improve representation quality for cross view similarity tasks. That said, deployment constraints remain. Full-size transformer models may be costly for low-latency edge inference. Typical mitigations include quantization, pruning, distillation, and mixed-precision serving. This hybrid strategy can keep latency acceptable while preserving quality in difficult cases.

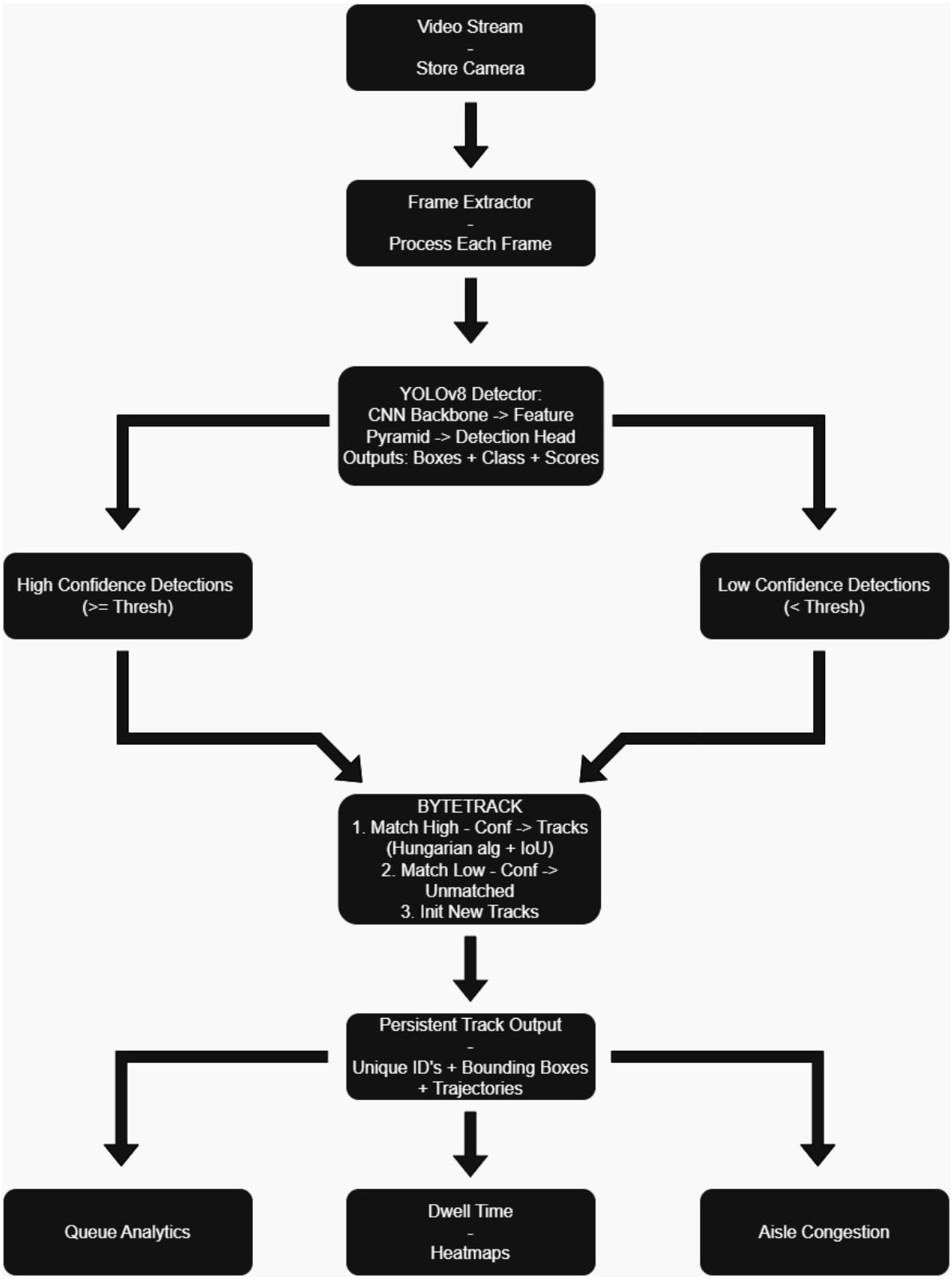
*Figure 5: ViT Pipeline*



### Technique 2: Real-time detection and tracking (YOLO plus ByteTrack):

For storing analytics and surveillance-style tasks, one of the most practical stacks is a real-time detector (for example, YOLO) paired with a tracking-by-detection algorithm such as ByteTrack [9], [10]. The detector provides per-frame boxes and classes; the tracker maintains identities across frames and enables trajectory-level features. ByteTrack is effective because it associates both high-confidence and low confidence detections under a principled matching strategy. This improves identity continuity in crowded scenes where many detectors would otherwise drop objects and fragment tracks. Retail use cases include queue length monitoring, dwell-time heatmaps, aisle congestion analysis, and conversion analytics (for example, product interaction without purchase). The main limitations are camera placement sensitivity and degraded identity consistency during severe occlusion. To address this, deployments usually incorporate zone constraints, temporal smoothing, and calibration procedures for each camera position.

*Figure 6: YOLOv8 + ByteTrack Pipeline*



## Two Creative & Novel Uses of CV Plus Audio Processing:

### Use Case 1: Accessibility assistant for elderly and low-vision shoppers:

A multimodal in-store assistant can combine computer vision scene understanding with speech interfaces to support independent shopping. Vision modules detect aisle markers, shelf sections, and candidate products; audio modules provide conversational guidance using ASR and TTS. Example flow: A shopper asks for “low-sodium soup”; the system identifies relevant shelf regions, then gives step-by-step directions and confirms candidate items by reading label highlights aloud. If the user grants consent, the system can maintain a session-level memory (“same brand as last time”) and propose alternatives when an item is out of stock. This use case addresses accessibility and inclusion while also reducing dependency on staff for routine navigation help.

### Use Case 2: Audiovisual checkout anomaly prevention:

Self-checkout issues often come from process mismatch rather than deliberate misuse. A fused CV + audio approach can detect these mismatches earlier and with fewer false alarms. Vision detects object transitions (basket to scan zone to bagging area), while audio detects scanner beep sequences and timing. If vision sees an item move to bagging but no corresponding beep event is detected within an expected time window, the system can trigger a soft prompt (“Please scan this item”) instead of an immediate hard intervention. Conversely, if a beep is detected without matching visual movement, the system can ask for a quick confirmation. Compared with vision-only systems, multimodal fusion is more robust when one signal degrades temporarily, such as partial camera occlusion or noisy acoustic environment. This enables assistive workflows that reduce friction and shrink shrinkage without over-penalizing legitimate customer behavior.

# References:


[1] Amazon, “Just Walk Out Technology,” Amazon.com, 2026.

[2] C. Goldman, A. Narayanan, and M. Batra, “AI in Retail: The Future of Shopping,” NVIDIA Developer Blog, 2024.

[3] W. Wei, S. Ren, B. Chen, et al., “RPC: A Large-Scale Retail Product Checkout Dataset,” in CVPR, 2019.

[4] A. Goldman, A. Herzig, J. Eisenschtat, et al., “Precise Detection in Densely Packed Scenes,” in CVPR (SKU-110K), 2019.

[5] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” in ICML, 2021.

[6] Y. Ge, R. Zhang, X. Wang, et al., “DeepFashion2: A Versatile Benchmark for Detection, Pose Estimation, Segmentation and Re-Identification of Clothing Images,” in CVPR, 2019.

[7] A. Dosovitskiy et al., “An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in ICLR, 2021.

[8] Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in ICCV, 2021.

[9] G. Jocher et al., “Ultralytics YOLO,” GitHub repository, 2023-2026.

[10] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in ECCV, 2022.

[11] N. Dalal and B. Triggs, “Histograms of Oriented Gradients for Human Detection,” in CVPR, 2005.

[12] R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022.

[13] OpenCV, "Video Motion Analysis and Object Tracking," OpenCV Documentation, 2026.

[14] OpenCV, "How to Use Background Subtraction Methods," OpenCV Documentation, 2026.

[15] OpenCV, "Other Tutorials," OpenCV Documentation, 2026.

[16] OpenCV, "Convex Hull," OpenCV Documentation, 2026.

[17] OpenCV, "Meanshift and Camshift," OpenCV Documentation, 2026.

[18] OpenCV, "Optical Flow," OpenCV Documentation, 2026.

[19] W3Schools, "Python Class __init__," W3Schools, 2026.

[20] FFmpeg Project, "ffprobe Documentation," FFmpeg, 2026.

[21] FFmpeg Project, "ffmpeg Documentation," FFmpeg, 2026.

[22] Python Software Foundation, "subprocess — Subprocess Management," Python 3 Documentation, 2026.

[23] Python Software Foundation, "json — JSON Encoder and Decoder," Python 3 Documentation, 2026.

[24] Python Software Foundation, "pathlib — Object-Oriented Filesystem Paths," Python 3 Documentation, 2026.

[25] Python Software Foundation, "math.gcd — Greatest Common Divisor," Python 3 Documentation, 2026.

[26] Chocolatey, "Chocolatey — The Package Manager for Windows," Chocolatey Software, 2026.

[27] P. K. Mishra and B. Banerjee, "Multiple Kernel Based KNN Classifiers for Vehicle Classification," International Journal of Computer Applications, vol. 71, no. 6, pp. 1–5, 2013.

[28] Y. Li et al., "Research on Vehicle Detection Based on Improved YOLOv8 Network," arXiv preprint arXiv:2501.00300, 2024.

[29] N. E. Indrayana, M. Sudarma, I K. G. D. Putra, and A. A. K. O. Sudana, "Comprehensive Analysis of YOLOv8 + DeepSORT for Vehicle Tracking," International Journal of Advanced Computer Science and Applications (IJACSA), vol. 16, no. 9, 2025.

[30] diagrams.net, "draw.io Diagramming Tool," diagrams.net, 2026.